

Why Random Attention Works.

Topic 1 made attention cheap. But cheap is useless if information cannot flow: can token 1 still influence token 4096? View the attention pattern as a *graph*. Dense attention is the complete graph — everyone reachable in one hop. A few *random* edges per node turn the sparse graph into an *expander*, whose diameter is $O(\log T)$, so global mixing takes only $\sim \log T$ layers. Every number below is computed in full.

THE CLAIM

local only: $\#\text{layers} \sim \frac{T}{w}$ (linear) $\implies$ local + random: $\#\text{layers} \sim \log_r T$ (logarithmic)

Worked example. $T = 4096$, $r = 3$ random neighbours: $\log_3 4096 \approx 7.6$, so ~ 8 layers suffice for any token to reach any other — versus ~ 9 for local-only at $T=4096$, blowing up to 257 at $T=131072$.

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	Attention as a graph	2
2	Local-only graphs mix slowly	3
3	Random edges make an expander	3
4	Worked computation: hops to mix	4
5	A random graph, drawn out ($T = 8$)	4
6	BigBird's theory & a warning	4
7	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 Attention as a graph

Build a graph $G = (V, E)$ with one node per token, $V = \{1, \dots, T\}$, and a (directed) edge $i \rightarrow j$ whenever the mask allows query i to attend to key j , i.e. $M_{ij} = 1$. One attention layer lets information travel one hop along these edges. After L stacked layers, node i has been influenced by every node reachable from i within L hops.

$$\boxed{\text{\#layers for global mixing} \geq \text{diam}(G)} \quad (1)$$

where the *diameter* $\text{diam}(G) = \max_{u,v} \text{dist}(u,v)$ is the longest shortest-path between any two nodes. If two tokens are k hops apart, they cannot exchange information in fewer than k layers. So the design question is precisely: *what is the diameter of the sparse attention graph?*

Each layer is one “relay”. A rumour started at token i spreads to its neighbours in layer 1, their neighbours in layer 2, and so on. The diameter is the worst-case number of relays needed for the rumour to reach everyone. Dense attention is gossip in a room where everyone hears everyone — diameter 1. We want a sparse seating chart that still spreads rumours fast.

Dense attention = complete graph. Every node is connected to every other: $|E| = T(T-1)$ edges, $\text{diam} = 1$. Perfect mixing, quadratic cost — exactly the trade-off Topic 1 attacked.

2 Local-only graphs mix slowly

A pure sliding window of width w connects i only to $\{i - w/2, \dots, i + w/2\}$. This is (essentially) a path/ring graph with short-range chords. Each layer extends a token's reach by $\leq w/2$ on each side, so to span the whole sequence requires

$$\#layers_{\text{local}} \approx \frac{T}{w} \quad (2)$$

layers — *linear* in T . The diameter of a window graph is $\Theta(T/w)$: opposite ends of a long document are physically far apart on the graph, and the only way across is to walk the local band one window at a time. Stacking 48 layers reaches at most $\approx 48 \cdot w/2$ positions; for $T = 131072$, $w = 512$ that is nowhere near enough.

3 Random edges make an expander

Now add r random neighbours per node. The headline facts from spectral graph theory:

1. A random d -regular graph ($d \geq 3$) is, with high probability, an **expander**: every subset S with $|S| \leq T/2$ has many edges leaving it, $|\partial S| \geq c|S|$ for a constant $c > 0$ independent of T . No bottlenecks.
2. Equivalently it has a bounded **spectral gap**. Let λ_2 be the second-largest eigenvalue of the adjacency matrix (largest is d). Expansion $\Leftrightarrow \lambda_2 \leq d - \varepsilon$ for fixed $\varepsilon > 0$; the Alon–Boppana bound gives $\lambda_2 \gtrsim 2\sqrt{d-1}$, and random graphs nearly achieve it (Ramanujan). A large gap $d - \lambda_2$ means fast mixing.
3. Consequently the **diameter is logarithmic**: a d -regular expander on T nodes has

$$\text{diam}(G) = O(\log_{d-1} T).$$

Intuitively each hop multiplies the reachable set by $\approx d - 1$, so $(d-1)^k \geq T$ after only $k = \log_{d-1} T$ hops.

$$\#layers_{\text{local+random}} \approx \log_r T \quad (3)$$

Local edges give resolution; a handful of *random* edges give reach. With $r = O(1)$ random neighbours the attention graph becomes an expander, collapsing the mixing depth from T/w layers down to $\log_r T$ layers. This is why BigBird needs only a constant number of random links to behave, for information flow, almost like dense attention.

4 Worked computation: hops to mix

With $r = 3$ random neighbours, the expander estimate is $\log_3 T$:

$$\begin{aligned} T = 512 : \quad \log_3 512 &= \frac{\ln 512}{\ln 3} = 5.68 \rightarrow 6 \text{ layers,} \\ T = 4096 : \quad \log_3 4096 &= \frac{\ln 4096}{\ln 3} = 7.57 \rightarrow 8 \text{ layers,} \\ T = 131072 : \quad \log_3 131072 &= \frac{\ln 131072}{\ln 3} = 10.73 \rightarrow 11 \text{ layers.} \end{aligned}$$

Contrast the *local-only* depth $\lceil (T-1)/(w-1) \rceil$ with $w = 512$:

T	local-only $\approx T/w$	local+random $\approx \log_3 T$	verdict
512	1	6	both fine (short)
1 024	3	7	comparable
4 096	9	8	expander wins
16 384	33	9	expander wins big
65 536	129	11	local hopeless
131 072	257	11	local hopeless

The two columns cross near $T \approx 4096$. Beyond it, local-only depth explodes *linearly* (257 layers for 128k tokens — no real transformer is that deep), while the expander depth crawls up *logarithmically*, gaining just one layer when T grows from 65k to 131k.

5 A random graph, drawn out ($T = 8$)

A random 3-regular graph on 8 nodes (configuration model, seed 20). Symmetric adjacency A ; **indigo** = edge present, light = none. Every node has exactly 3 neighbours.

	0	1	2	3	4	5	6	7
0	0	1	0	0	0	1	0	1
1	1	0	1	1	0	0	0	0
2	0	1	0	0	1	0	0	1
3	1	1	0	0	0	0	1	0
4	0	0	1	0	0	1	1	0
5	1	0	0	0	1	0	1	0
6	0	0	0	1	1	1	0	0
7	1	0	1	1	0	0	0	0

This graph has 12 edges and **diameter 2**: every pair of the 8 nodes is joined by a path of at most 2 hops. The prediction $\lceil \log_3 8 \rceil = \lceil 1.89 \rceil = 2$ matches exactly. Two layers of this sparse attention already mix the whole sequence — compare a 2-regular ring on 8 nodes, whose diameter would be 4.

6 BigBird’s theory & a warning

1. **Expressivity is preserved.** BigBird (local + global + random) is a *universal approximator* of sequence-to-sequence functions and is *Turing-complete*, matching full attention — the global tokens supply the long-range “registers” and the expander structure supplies the routing. Sparsity costs cheapness, not power.

2. **Global tokens are diameter-2 hubs.** A single global token alone gives diameter ≤ 2 (everyone $\rightarrow$ hub $\rightarrow$ everyone), but routing *everything* through one node is a fragile bottleneck; random edges spread the load and keep the spectral gap healthy.
3. **Depth budget.** Real models have $L \approx 12\text{--}48$ layers. Equation (3) says this comfortably covers $\log_3 T$ for T into the hundreds of thousands — the regime where dense attention is impossible.

Too few random edges and the guarantee evaporates. With $r = 1$ the graph need not even be connected: it can split into components that *never* exchange information, no matter how many layers you stack (diam = ∞). The expander/ $\log T$ -diameter result requires $d \geq 3$ (counting all edge types). Skimp on randomness and you get isolated islands of tokens, not a sequence-wide mixer.

7 Where this sits in the track

Direction	Topic	Link
Back	Sparse local+global+random	gave us the cheap mask
Here	Expander-graph mixing	proves the mask still mixes in $O(\log T)$ layers
Next	Phase 6: linear-attention kernels	a different route to $O(T)$

A Reproducible (NumPy)

Inputs: $w = 512$, $r = 3$; random 3-regular graph on $T = 8$ (configuration model, seed 20).

```
import numpy as np, math, collections
w, r = 512, 3
for T in [512,1024,4096,16384,65536,131072]:
    local = math.ceil((T-1)/(w-1)) # local-only depth
    expander = math.ceil(math.log(T)/math.log(r)) # log_r(T)
    print(T, local, expander)
# 512 1 6 ; 4096 9 8 ; 65536 129 11 ; 131072 257 11

# random 3-regular graph on 8 nodes via configuration model
rng = np.random.default_rng(20)
def rand_reg(n,d):
    while True:
        stubs = list(range(n))*d; rng.shuffle(stubs)
        A = np.zeros((n,n),int); ok = True
        for k in range(0,len(stubs),2):
            a,b = stubs[k],stubs[k+1]
            if a==b or A[a,b]: ok=False; break
            A[a,b]=A[b,a]=1
        if ok: return A
A = rand_reg(8,3)
print(A.sum()//2, A.sum(1)) # 12 edges, degree 3 everywhere
```

```
def diameter(A):                                # BFS from every node
    n=len(A); best=0
    for s in range(n):
        dist=[-1]*n; dist[s]=0; q=collections.deque([s])
        while q:
            u=q.popleft()
            for v in range(n):
                if A[u][v] and dist[v]<0: dist[v]=dist[u]+1; q.append(v)
            best=max(best,max(dist))
    return best
print(diameter(A), math.ceil(math.log(8,3)))    # 2  2
```

Marevlo Research · Transformer Track · Module 5.2 · Topic 2 of 2.